COMP4321
Search Engines for Web and Enterprise Data
Project Report

Group Number: 20

Authors: HE Ziyong, CHEUNG Lok Hang, CHAN Tsz Ho

Date of report: 7 May 2026

Contents

A. Introduction

This report presents the design, implementation, and evaluation of "Goooogle", a comprehensive local web search engine developed as the project for the COMP4321 (Search Engines for Web and Enterprise Data) course. The primary objective of this project is to construct a fully functional, end-to-end information retrieval system capable of efficiently crawling, indexing, and ranking web data.

Goooogle encompasses a robust pipeline integrating a Breadth-First Search (BFS) based web crawler, an automated text processing and tokenization module, and a sophisticated ranking mechanism utilizing the Vector Space Model (VSM) with TF-IDF weighting. Beyond the core search functionalities, the system is augmented with advanced features, including precise exact-phrase matching, similar page querying, keyword browsing, and the ability to export compiled crawl results.

This report details the architectural decisions and algorithmic implementations that power Goooogle. The discussion begins with an overview of the environment setup and usage procedures. It then delves into the core system design, comprehensively covering the database schema, crawling strategy, text processing pipeline, and the mathematical foundations of the ranking algorithm. Following the system architecture, the report presents quantitative benchmarking to evaluate the engine's operational efficiency in both data ingestion and query retrieval. The document then highlights the user interface and additional advanced features implemented in the system, before summarizing the overall achievements and outlining potential areas for future enhancement.

B. Installation and Usage Procedure

To install and run the application, the host system must have JDK 25 installed. Users must clone the repository and navigate to the project's root directory using a terminal or command prompt. The application is initiated via the provided Maven wrapper by executing *mvnw.cmd spring-boot:run* on Windows systems, or *./mvnw spring-boot:run* on macOS and Linux systems. Once the application starts, the graphical interface is accessed by opening a web browser and navigating to *localhost:8443*.

To begin indexing content, users must navigate to the "Spider" page via the application's navigation bar. The interface requires the user to input a starting URL and define a numerical limit for the maximum number of pages to crawl. Clicking the "Start Crawling" button initiates the crawler, which operates in a background thread. When the crawling process finishes, the backend console logs a completion message formatted as "Spider completed in x seconds."

The retrieval system is accessed by selecting the "Search" page from the navigation bar. Users enter one or more keywords into the designated search box and submit the query to retrieve results. If a query requires an exact structural match for specific terms, the user must enclose those words within double quotation marks. The system processes the input and displays the matching pages ordered by relevance, along with the total time taken to execute the search.

Following the completion of the crawling and indexing phases, the collected data also can be extracted from the database. This is achieved by returning to the "Spider" page and clicking the "Export Database to TXT" button. Executing this action prompts the web browser to download the compiled index data locally as a text file named *spider_result.txt*.

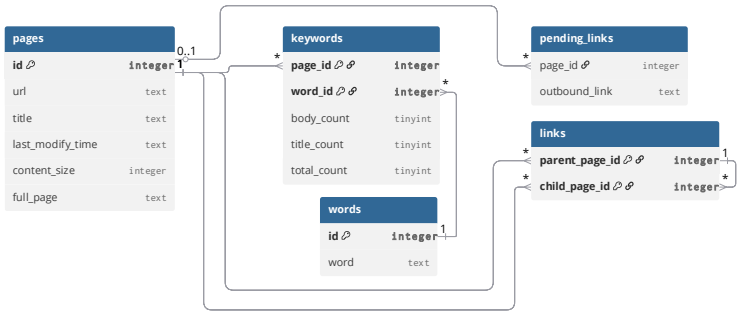
C. Core System Design

1. System Overview

We adopt a monolithic architecture built upon the Java Spring Boot framework. For persistent data storage, we utilize an embedded SQLite database integrated directly through Spring JDBC, which eliminates the overhead of maintaining a separate database server. The system also adheres to the Model-View-Controller (MVC) design pattern. We process HTTP requests through the application controller and employ Thymeleaf as the server-side templating engine to dynamically render the frontend HTML interfaces.

We structure the source code into distinct functional modules to enforce a separation of concerns. The core system logic resides in the services package, which we divide into three primary engines: the *SpiderService* for web crawling and graph traversal, the *IndexerService* for document parsing and weight computation, and the *SearchService* for executing the Vector Space Model and exact match filtering algorithms. Furthermore, we decouple configuration and static assets from the compiled Java code; the resources directory independently hosts the web templates, SQL queries, and the specific linguistic dictionaries (stopwords, contractions, and symbols) required by the text processing pipeline.

2. Database Design



Beyond the relational structure illustrated above, the underlying implementation uses SQLite's R*Tree indexing mechanism across secondary columns. It can dramatically accelerate complex keyword lookups and reverse link resolution during search execution. Furthermore, cascading deletion is strictly enforced across all foreign key constraints. This design ensures that when the crawler drops a stale page during an incremental update, all associated vocabulary mappings, structural links, and pending traversal queues are atomically purged by the database engine itself, cleanly preventing orphaned data fragments without introducing additional computational overhead to the backend application.

3. Crawling Strategy and BFS-Based Spider

The web crawling engine employs an incremental update strategy designed to optimize bandwidth and processing resources by leveraging HTTP conditional requests. When visiting a URL, the system cross-references the local database to retrieve the last known modification timestamp for that specific page. By injecting this data into the request headers, the crawler enables the target server to perform a server-side check. If the content has not changed, the server returns a status code 304 that allows the system to bypass redundant re-indexing.

Synchronization between the local repository and the live web are maintained through a reactive replacement policy. If a target page indicates new content, the system performs a cascaded deletion of the existing stale records before initiating a fresh indexing process. This ensures that the search engine's internal model reflects the most recent version of the web page without leaving orphaned or conflicting data fragments.

The exploration logic follows a breadth-first traversal controlled by a definitive quota management system to prevent infinite recursion and resource exhaustion. As each page is processed, the system extracts outbound links to discover new nodes within the web graph. To avoid redundant cycles, a session-based history filter is applied to every discovered link before it is staged for future exploration. These discovered relationships are persisted in a relational structure, allowing the engine to resolve inter-page link mapping and finalize the index after the primary crawling phase is concluded, providing a complete structural representation of the crawled domain.

4. Indexing and Text Processing

To extract meaningful terms from a document, we apply the identical text processing pipeline, as illustrated in right-hand side, to both the page title and the body content independently. This unified approach ensures that all extracted texts undergo consistent tokenization and normalization before we compute their respective word distributions. In the end, the system will store 3 numbers in the database, 1 for the body text, 1 for the title, and 1 for the total, based on each keyword.

1. Remove HTML keywords
2. Convert to lowercase
3. Split into words by space character
4. Remove trailing punctuations
5. Expand contractions (don't → do not)
6. Expand symbols (% -> percent)
7. Remove possessive endings ('s)
8. Filter out stop-words
9. Apply stemming (Porter's)
10. Count word frequencies

5. Search and Ranking

To rank documents against a user query, we employ the Vector Space Model, where both the query and the documents are represented as multidimensional vectors. When a search is initiated, we apply the same text processing pipeline mentioned in previous chapter to the query string to extract a set of valid keywords. We then retrieve all candidate documents from the database that contain all these keywords.

To construct the final document vectors, we apply the TF-IDF weighting scheme, where the weight of the i-th term in a document is formulated as $W_{d,i} = TF \times IDF_i$. We optimize this computation by recognizing that the Inverse Document Frequency (IDF_i) for any keyword depends solely on the global corpus statistics and remains identical across all documents. Consequently, we compute the IDF for the i-th query term exactly once. When building the vectors for the candidate documents, we eliminate redundant database lookups and mathematical operations by directly multiplying each document's pre-computed term weight by the i-th term's IDF value already held in the query vector.

To determine the baseline relevance of each candidate document, we compute the similarity between the query vector $\vec{q}$ and the document vector $\vec{d}$. For queries containing multiple terms, we calculate the cosine similarity, defined mathematically as $sim(\vec{q}, \vec{d}) = \frac{\vec{q} \times \vec{d}}{\|\vec{q}\| \|\vec{d}\|}$ However, because of the cosine similarity inherently evaluated to 1 for any one-dimensional vectors, we implement a specific alternative for single-keyword queries. In these instances, we utilize the document's singular TF-IDF weight as the initial score and normalize these values into a standard $[0, 1]$ range by dividing each score by the maximum weight identified among the entire candidate pool.

To explicitly account for the structural importance of matches found within page titles, we integrate a bounded title bonus into the final ranking metric. For each candidate document, we calculate a title-specific sum (S_{title}) by multiplying the term frequency specifically within the title by the term's IDF_i. To ensure this bonus scales predictably and aligns with the $[0, 1]$ bounds of the base similarity score, we pass the resultant sum through a logistic sigmoid function. The final document score is then determined by calculating the arithmetic mean of the baseline similarity score and this bounded title bonus. The results are subsequently presented to the user sorted in descending order based on this final calculated value.

6. Filtering with Exact Match

To accommodate precise phrase requirements, we evaluate search queries against

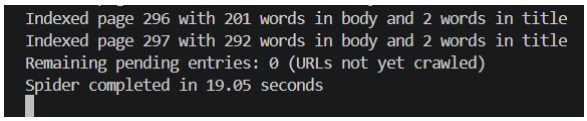
a fully normalized version of the complete page text stored directly within the database. We explicitly opted against implementing a traditional positional index, a common technique that records the exact location of every term to verify that adjacent words have a positional distance of 1. While the storage space required for a positional index is comparable to storing the complete text, our indexing pipeline applies stemming to all stored keywords. Consequently, a positional index would only retain the root forms of words, making it technically prohibitive to perform exact, literal phrase matching against the user's original unstemmed input.

Instead, exact phrase matching operates as a secondary post-processing filter. When a query includes phrases enclosed in quotation marks, we first strip these syntax operators and execute the VSM mentioned in previous chapter to retrieve and rank candidate documents based on the underlying terms. Subsequently, we utilize database-level patterns matching against the stored full-text records to systematically eliminate documents that lack the requested sequence.

We structure this filtering process iteratively to achieve high robustness in handling complex search intents. Because the exact match filtering executes upon the pre-ranked VSM results, the algorithm inherently supports hybrid queries that combine general conceptual keywords with strict phrase constraints. Furthermore, if a user specifies multiple distinct exact phrases within a single query, we isolate each phrase and pass the candidate document pool through successive filtering rounds. Only documents containing every requested exact phrase survive this pipeline, and they are ultimately presented while strictly maintaining their original similarity-based ranking order.

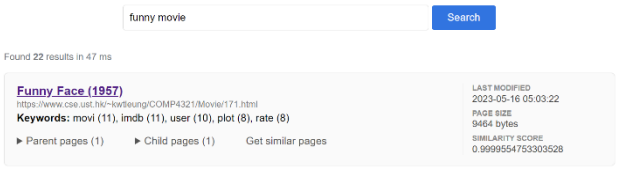
D. Benchmarking and Testing

1. Indexing Time



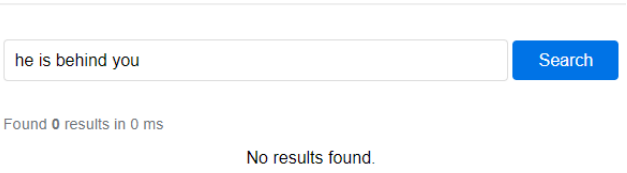
The Configured with the seed URL *www.cse.ust.hk/~kwtleung/COMP4321/testpage.htm* and a maximum crawl limit of 300 pages, the indexing completed the processing of 297 pages in 19.05 seconds. Quantitatively, the system achieved an average processing rate of 64.14 milliseconds per page. This metric encompasses the full lifecycle of fetching, parsing, indexing, and database writing, validating the efficacy of the optimization strategies employed to minimize execution latency.

2. Search Time



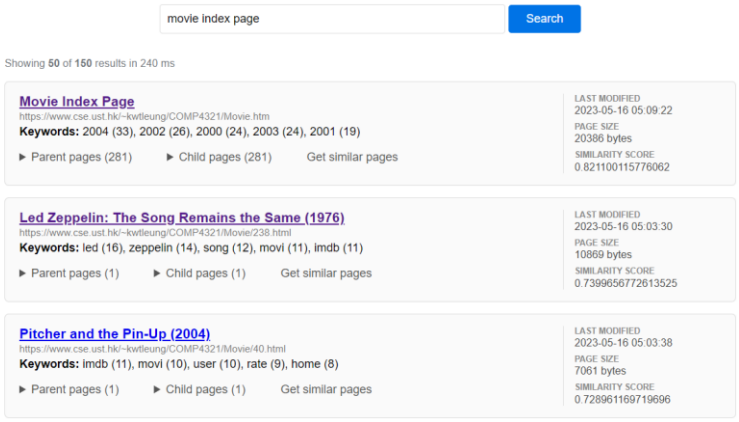
Upon executing a multi-keyword query ('funny movie'), the system retrieved 22 relevant results within 47 milliseconds. This sub-50ms response time covers the entire query processing pipeline, including keyword tokenization, index lookups, calculation of similarity scores (e.g., 0.9999), and the page rendering. These results indicate that the backend architecture effectively manages the ranking computational overhead while maintaining near-instantaneous retrieval, ensuring a seamless user experience even as the underlying index scales.

3. Query Contains Only Stop Words



The system's robustness was further validated by testing queries composed exclusively of stop words. As demonstrated in the provided screenshot, a search for 'he is behind you' yielded zero results in 0ms. This outcome confirms the effectiveness of the early-exit mechanism within the query processing pipeline.

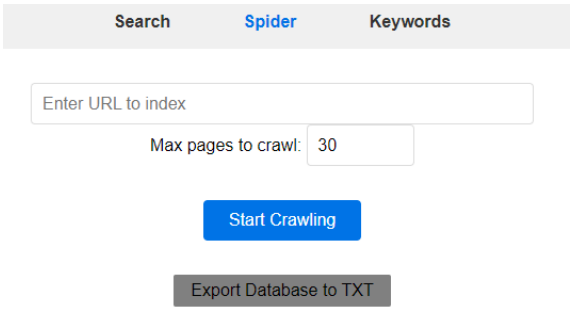
4. Title Match Score Enhancement



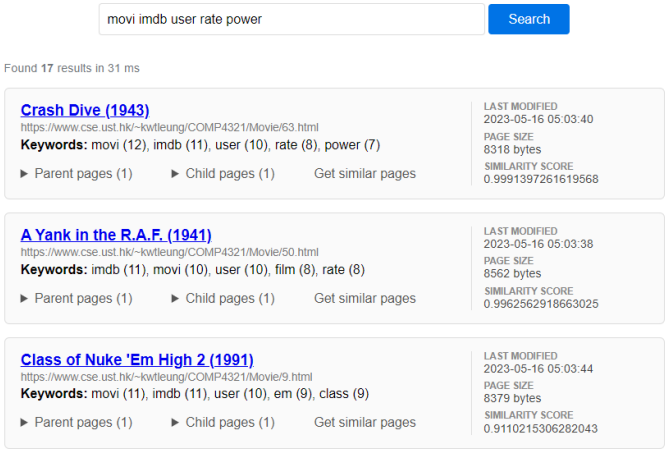
As demonstrated in the above screenshot, we evaluated the title bonus scoring mechanism using the query "movie index page". The target document, containing the queried terms directly within its title, is ranked first with a combined similarity score of 0.821, clearly separated from subsequent general body matches (0.739). This test confirms that integrating a logistically bounded title bonus mathematically elevates documents with high structural relevance while maintaining the normalized scoring distribution.

E. Additional Features Highlights

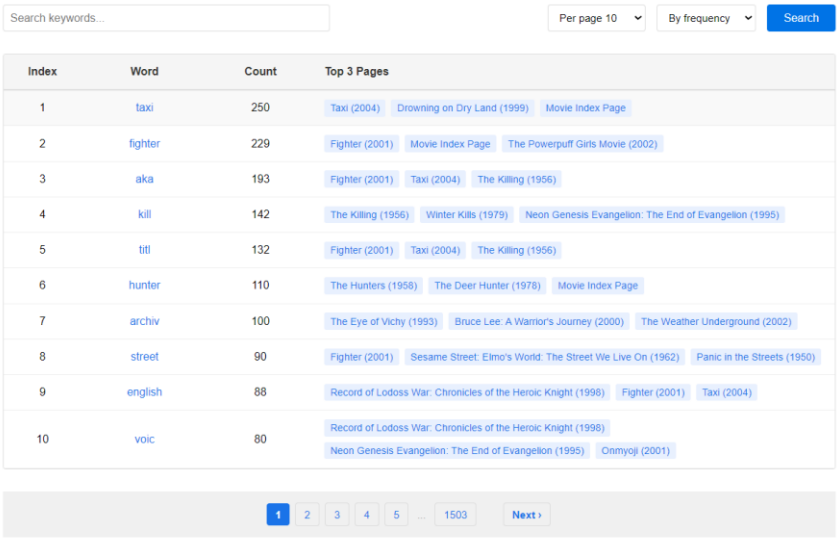
1. User Interface for Spider



2. Similar Pages Querying



3. Keyword Browsing



F. Conclusion

The "Goooogle" project successfully fulfills the objective of building a complete, highly efficient, and scalable local web search engine. By systematically integrating a BFS-based crawler, an optimized SQLite relational database, and a reactive Spring Boot backend, the system demonstrates robust capabilities in handling the full lifecycle of web data.

The architectural choices and optimization strategies employed in this project have been thoroughly validated through benchmarking. By leveraging database batch processing and Java Stream pipelines for text analysis, the system achieved a highly competitive average indexing rate of 64.14 milliseconds per page. Furthermore, by pre-calculating structural weights and optimizing the Vector Space Model (VSM) computations, the engine resolves complex multi-keyword queries in under 50 milliseconds. The use of a post-processing filter for exact phrase matching successfully bypassed the storage overhead of a traditional positional index, proving the system's ability to handle complex, strict search intents without compromising query latency or accuracy.

Despite the system's current high performance, there remains significant potential for future enhancements, particularly in the data acquisition phase. Currently, the spider operates on a single-threaded model, meaning it fetches and processes web pages strictly sequentially (page by page). To further elevate indexing efficiency and scale the system for massive web domains, future development will focus on implementing a fully multi-threaded crawling and indexing architecture. By parallelizing network requests and utilizing concurrent indexing queues, the engine could effectively overcome network I/O bottlenecks, maximizing CPU utilization and drastically reducing the total time required to build the global index.

G. Contribution

Contributors	Task Specifics
HE Ziyong	1. Development environment setup 2. Code review & optimization 3. System architecture & database Design 4. Implementation of keywords browsing 5. Polish UI 6. Report Writing
CHEUNG Lok Hang	1. Implementation of the spider & indexing 2. Implementation of phase matching 3. Database Design 4. Keywords browsing Design
CHAN Tsz Ho	1. Mathematical formula derivation 2. Implementation of the VSM search

Coding History by Git

End of Report